connection explicitly unsubscribes. A more complex channel might have some logic in those methods. In our case, all we need to do is use the subscribed method to attach the new subscriber to a stream using the stream_from method. The argument to stream_from is the name of the stream, which we'll use to guide future broadcasts. In our case, the name schedule is a static literal; however, in other cases, the name might dynamically depend on an incoming parameter when the subscription is made. (Our next example from our concert show page, for instance, will need to specify which concert is being subscribed to.) Not all channel classes are this small; channels that expect to receive data from the client will add methods to respond to those requests.

We can send data to this stream from anywhere in our Rails program by calling the method ActionCable.server.broadcast with the first argument being the name of the stream to broadcast to and the second argument being the data to send.

Because you can add that broadcast statement anywhere, we now need to decide where to add it. If this sample code were a real program, what we'd likely do after every ticket purchase is check to see if the purchase results in a newly sold-out concert and, if so, broadcast the message. However, this sample code isn't a real program, and we don't really have ticket sales built in. So for the moment, we're going to repurpose the SoldOutConcertsController that we used earlier to send JSON data:

chapter_09/03/app/controllers/sold_out_concerts_controller.rb

```
class SoldOutConcertsController < ApplicationController
  def show
    concerts = Concert.includes(:venue, gigs: :band).all
    sold_out_concert_ids = concerts.select(&:sold_out?).map(&:id)
»   ActionCable.server.broadcast(
»     "schedule",
»     {soldOutConcertIds: sold_out_concert_ids}
»   )
    render(json: {soldOutConcertIds: sold_out_concert_ids})
  end
end
```

The key line here is ActionCable.server.broadcast("schedule", soldOutConcertIds: